

Milestone 4 Report

TrentoParking

Team 12

4 Giugno 2026

Contents

1	Sezione Introduttiva	2
1.1	Team Members	2
1.2	Project Idea	2
1.3	Links	2
2	Sezione Generale	2
2.1	Strategia di Branching	2
2.2	Product Backlog	3
2.3	Definizione di Done	4
3	Sprint 2	5
3.1	Goal	5
3.2	Sprint Planning	5
3.2.1	Burndown Chart	7
3.3	Test Cases	7
3.4	Sprint Review	9
3.5	Product Backlog Refinement	9
3.6	Sprint Retrospective	9
3.6.1	Aspetti positivi	10
3.6.2	Criticità riscontrate	10
3.6.3	Pratiche Agile adottate	10
3.6.4	Design Thinking	10
3.6.5	Dinamiche di gruppo	11
4	Sezione Finale	11
4.1	Diagramma del Deploy	11
4.2	Stack Tecnologico	12
4.3	Conclusioni	12

1 Sezione Introduttiva

1.1 Team Members

Nome e Cognome	Matricola	Account Github
David Dorobantu	234467	https://github.com/DavidDorobantu
Riccardo Gonzato	246476	https://github.com/Gonzo04
Matteo Sepa	243283	https://github.com/sepamatteo

Table 1: Membri del team

1.2 Project Idea

TrentoParking è un'applicazione web dedicata al noleggio di posti auto privati. L'obiettivo del progetto è permettere ai proprietari di parcheggi privati di mettere a disposizione il proprio posto auto quando non utilizzato, mentre gli utenti possono cercare e prenotare rapidamente un parcheggio disponibile tramite una mappa interattiva. L'applicazione vuole ridurre il tempo necessario alla ricerca di parcheggio, migliorare la mobilità urbana e ottimizzare l'utilizzo degli spazi privati inutilizzati.

1.3 Links

- Repository Github: <https://github.com/Gonzo04/TrentoParking>
- Link Apiary: <https://trentoparking.docs.apiary.io/#>
- Link deploy: <https://trentoparking-frontend.onrender.com>
- Account demo:
 - **Admin** — email: `admin@trentoparking.it`, password: `admin123`
 - **Host** — email: `host@trentoparking.it`, password: `host123`
 - **Utente** — email: `mario.rossi@gmail.com`, password: `password123`

2 Sezione Generale

2.1 Strategia di Branching

Il team ha mantenuto la strategia di branching basata sul modello *GitFlow semplificato senza develop* adottata nel primo sprint. I branch utilizzati sono:

- **main**: contiene esclusivamente versioni stabili e funzionanti dell'applicazione.
- **feature/***: branch dedicati allo sviluppo di specifiche funzionalità.

Rispetto al primo sprint, nel secondo sprint il team ha lavorato in parallelo su un numero maggiore di branch contemporaneamente, integrando le modifiche tramite Pull Request su GitHub prima di ogni merge in **main**. I principali branch del secondo sprint sono stati:

- `feature/spot-photos` — caricamento foto per i posti privati
- `feature/chat` — sistema di messaggistica tra utente e host
- `feature/review` — sistema di recensioni dei posti privati
- `feature/admin-dashboard` — pannello di amministrazione
- `feature/booking-safe-spot-status` — protezione eliminazione posti con prenotazioni future attive
- `refactor/frontend-structure` — riorganizzazione della struttura del frontend in `pages/`, `features/` e `components/common/`

Il flusso di lavoro ha previsto: sviluppo su branch dedicato → apertura Pull Request → revisione del codice → merge in `main`. Questa procedura ha permesso di rilevare e risolvere i conflitti in modo controllato, in particolare in occasione del refactoring del frontend.

2.2 Product Backlog

Di seguito il Product Backlog aggiornato con le user story introdotte nel secondo sprint (ID 13–17).

ID	Name	User story	Imp.	How to Demo
1	Registrazione	Come utente voglio registrarmi tramite email e password.	100	Account creato e email di verifica ricevuta.
2	Verifica Email	Come utente voglio verificare il mio account tramite email.	60	Click sul link: account attivato.
3	Login	Come utente voglio effettuare il login per accedere alla piattaforma.	100	Credenziali corrette: redirect alla home.
4	Password reset	Come utente voglio recuperare la password dimenticata.	60	Email di reset ricevuta, password aggiornata.
5	Mappa interattiva	Come utente voglio visualizzare una mappa per trovare parcheggi.	60	Mappa centrata su Trento con marker parcheggi.
6	Visualizzazione parcheggio	Come utente voglio vedere i posti disponibili sulla mappa.	80	Marker caricati dinamicamente dal backend.
7	Filtro per raggio	Come utente voglio filtrare i parcheggi per distanza.	30	Slider raggio: mappa filtra i posti.

ID	Name	User story	Imp.	How to Demo
8	Menu parcheggi card	Come utente voglio vedere i parcheggi in una lista laterale.	80	Sidebar sincronizzata con la mappa.
9	Selezione dalla mappa	Come utente voglio selezionare un parcheggio dalla mappa o dalla lista.	50	Selezione marker: dettagli del posto.
10	Prenotazione	Come utente voglio prenotare un posto auto disponibile.	100	Prenotazione confermata con pagamento mock.
11	Recensioni	Come utente voglio lasciare recensioni ai proprietari dei parcheggi.	60	Recensione inserita dopo prenotazione conclusa.
12	Messaggi	Come utente voglio comunicare con il proprietario del parcheggio.	70	Chat aperta: messaggi inviati e ricevuti.
13	Foto posti	Come host voglio caricare foto del mio posto per renderlo più attrattivo.	30	Foto caricate dall'host visibili nel dettaglio posto.
14	Gestione posti host	Come host voglio gestire i miei posti (pubblicazione, modifica, eliminazione).	60	Host modifica ed elimina i propri posti dalla dashboard.
15	Prenotazioni ricevute	Come host voglio visualizzare le prenotazioni ricevute sui miei posti.	50	Host accede alla sezione e vede dettagli e stato.
16	Profilo utente	Come utente voglio modificare i miei dati personali.	40	Utente aggiorna nome e targa dalla pagina profilo.
17	Pannello admin	Come amministratore voglio gestire utenti, posti e prenotazioni.	40	Admin modifica ruoli e annulla prenotazioni dal pannello.

Table 2: Product Backlog aggiornato

2.3 Definizione di Done

Una funzionalità viene considerata *Done* quando soddisfa tutti i seguenti criteri (invariata rispetto allo Sprint 1):

- il codice è stato implementato e caricato su Github;
- il codice compila senza errori;

- sono stati eseguiti test manuali e/o automatici;
- il codice è stato revisionato da almeno un altro membro del team;
- non sono presenti bug critici noti;
- la user story associata soddisfa i requisiti definiti durante lo sprint planning.

3 Sprint 2

3.1 Goal

L'obiettivo principale dello Sprint 2 è stato completare le funzionalità core dell'applicazione, trasformando TrentoParking da un prototipo di autenticazione e visualizzazione mappa in un sistema di prenotazione parcheggi completo e multi-ruolo.

Durante questo sprint il team si è concentrato su:

- completamento e consolidamento del sistema di prenotazione con pagamento;
- implementazione del sistema di recensioni per i posti prenotati;
- caricamento e visualizzazione di foto per i posti privati;
- sistema di chat tra utente e host;
- gestione del profilo utente;
- dashboard host per la gestione dei propri posti e delle prenotazioni ricevute;
- pannello di amministrazione per la gestione di utenti, posti e prenotazioni;
- refactoring della struttura frontend per migliorare la manutenibilità.

3.2 Sprint Planning

Durante lo Sprint Planning il team ha analizzato le user story prioritarie e stimato il carico di lavoro.

	User Story	Task	Volunt.	Est.
Sprint #2	Come utente voglio prenotare un posto auto disponibile e gestire le mie prenotazioni.	Completamento API prenotazioni	Matteo	3
		UI calendario e form prenotazione	Matteo	4
		Sistema pagamento (mock)	Matteo	3
		Lista e cancellazione prenotazioni	Matteo	3
		Testing prenotazioni	Matteo	2
		Deploy servizio prenotazioni	Matteo	1

	User Story	Task	Volunt.	Est.
	Come host voglio caricare foto del mio posto.	API upload foto (Multer)	Matteo	3
		Integrazione upload frontend	Matteo	3
		Visualizzazione foto nel dettaglio	Matteo	2
	Come utente voglio lasciare recensioni ai proprietari.	Modello e API recensioni	David	3
		UI modale recensione e stelle	David	3
		Calcolo e visualizzazione media	David	3
		Testing recensioni	David	2
	Come utente voglio comunicare con il proprietario del parcheggio.	Modello e API messaggi	Matteo	3
		UI ChatModal con polling	Matteo	4
		Integrazione chat nelle prenotazioni	Matteo	2
	Come utente voglio modificare i miei dati personali.	API modifica profilo	Riccardo	2
		UI pagina profilo	David	4
	Come host voglio gestire i miei posti auto.	API CRUD posti host	Riccardo	4
		UI dashboard gestione posti	Riccardo	4
		Blocco eliminazione posti con prenotazioni	Riccardo	2
	Come host voglio visualizzare le prenotazioni ricevute.	API prenotazioni ricevute	Riccardo	3
		UI lista prenotazioni ricevute	Riccardo	3
	Come amministratore voglio gestire utenti, posti e prenotazioni.	Backend API admin e middleware	Matteo	5
		UI AdminDashboard (3 tab)	Matteo	5
		Integrazione e testing admin	Matteo	2
	Refactoring e miglioramenti tecnici.	Refactoring struttura frontend	Riccardo	5
		Miglioramenti UI e stile	David	4
		Bug fix generali	David	4
Total			90	

	User Story	Task	Volunt.	Est.
--	------------	------	---------	------

Table 3: Sprint Backlog Sprint 2

3.2.1 Burndown Chart

Il Burndown Chart mostra l'andamento dello sprint e il completamento progressivo delle attività (totale: 90 story point, durata: 21 giorni). L'asse Y indica i punti rimanenti; l'asse X i giorni trascorsi dall'inizio dello sprint.

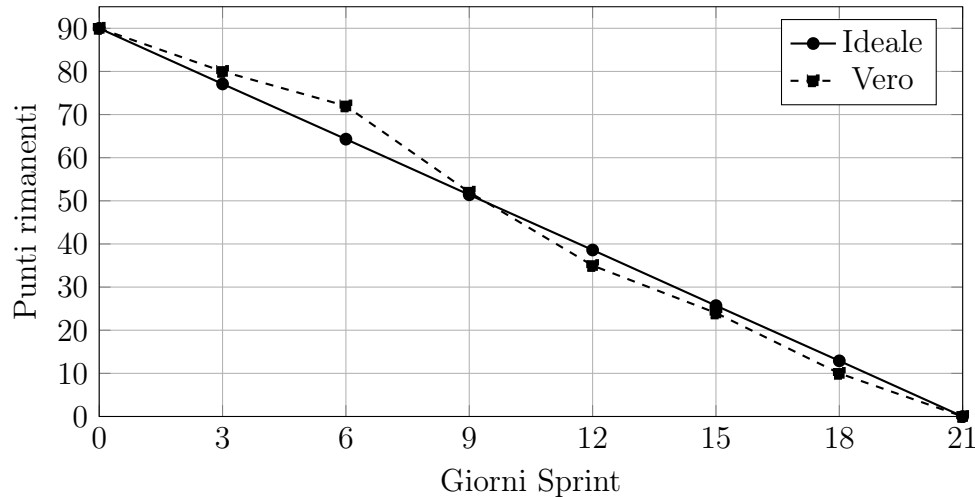


Figure 1: Burndown Chart Sprint 2

3.3 Test Cases

Di seguito sono riportati 20 test case relativi alle funzionalità implementate nello Sprint 2.

User Story	ID	Description	Test Case Data	Precond.	Expected Result
Prenotazione	01	Prenotazione riuscita	Slot libero, targa: AA000BB	Loggato	Prenotazione creata, stato IN_ATT_PAG.
	02	Slot già occupato	Slot già prenotato	Loggato	Errore disponibilità.
	03	Pagamento prenotazione	Click "Paga ora"	IN_ATT	Stato diventa PAGATA.
	04	Cancellazione prenotazione	Click "Annulla"	Prenotazione attiva	Stato diventa ANNULLATA.
	05	Lista prenotazioni utente	Accesso a "Le mie prenotazioni"	Loggato	Lista prenotazioni mostrata.
Foto posti	06	Upload foto valide	3 file JPG	Host loggato	Foto salvate e visibili nel dettaglio.

User Story	ID	Description	Test Case Data	Precond.	Expected Result
	07	Upload più di 10 foto	11 file selezionati	Host loggato	Errore: limite 10 foto.
	08	Visualizzazione foto	Apertura dettaglio posto	Foto caricate	Carosello immagini visibile.
Recensioni	09	Inserimento recensione valida	4 stelle, commento testuale	Prenotazione conclusa	Recensione salvata, media aggiornata.
	10	Recensione duplicata	Seconda recensione stessa prenotazione	Recensione già presente	Errore duplicato.
	11	Visualizzazione media stelle	Accesso scheda posto	≥ 1 recensione	Media e totale recensioni mostrati.
	12	Recensioni per host	Accesso pagina recensioni host	Recensioni in DB	Lista con stelle e commento.
Chat	13	Invio messaggio	Testo: "Posso parcheggiare alle 15?"	Prenotazione attiva	Messaggio inviato e visibile.
	14	Ricezione messaggi	Messaggio inviato dall'altro utente	Chat aperta	Messaggio appare entro 4 secondi.
	15	Accesso non autorizzato	Utente terzo chiama GET /chat/:id	Non coinvolto	Errore 403.
Gestione posti host	16	Pubblicazione nuovo posto	Nome, indirizzo, tariffa oraria	Host loggato	Posto creato e visibile sulla mappa.
	17	Modifica posto	Modifica tariffa oraria	Posto esistente	Tariffa aggiornata.
	18	Eliminazione con prenotazioni	Click "Elimina"	Prenotazioni future attive	Errore: eliminazione bloccata.
Pannello admin	19	Cambio ruolo utente	Ruolo: UTENTE $\rightarrow$ HOST	Admin loggato	Ruolo aggiornato in DB.
	20	Annullamento da admin	Click "Annulla" su prenotazione	Admin loggato	Stato diventa ANNULLATA.

Table 4: Test Cases Sprint 2

3.4 Sprint Review

Durante la Sprint Review il team ha effettuato una dimostrazione delle funzionalità implementate nel secondo sprint.

Sono state presentate:

- il flusso completo di prenotazione di un posto auto (ricerca, selezione, prenotazione e pagamento mock);
- il caricamento e la visualizzazione di foto associate ai posti privati;
- il sistema di recensioni con calcolo della media stelle per host e per posto;
- la chat tra utente prenotante e host con aggiornamento automatico;
- la dashboard host per la gestione dei posti pubblicati e la visualizzazione delle prenotazioni ricevute;
- la pagina di modifica del profilo utente;
- il pannello di amministrazione con gestione di utenti, posti e prenotazioni.

La discussione successiva si è concentrata su:

- miglioramenti all'esperienza utente nel flusso di prenotazione;
- possibili ottimizzazioni delle performance lato frontend;
- gestione dei casi limite nella chat (messaggi lunghi, connessioni lente).

3.5 Product Backlog Refinement

Durante il refinement il team ha rianalizzato il Product Backlog alla luce del lavoro svolto nel secondo sprint. Tutte le user story pianificate per Sprint 2 (ID 10–17) sono state completate e marchiate come *Done*: il sistema di prenotazione, le foto dei posti, le recensioni, la chat, la gestione del profilo, la dashboard host e il pannello admin sono stati consegnati e verificati secondo la Definizione di Done adottata dal team.

Il team ha inoltre rivalutato le priorità residue: poiché tutte le 17 user story del Product Backlog risultano completate, non sono state aggiunte nuove voci né modificate le stime esistenti. La discussione si è concentrata sulla qualità del prodotto finale, identificando come possibili aree di miglioramento futuro l'ottimizzazione delle performance del frontend e l'introduzione di notifiche in tempo reale per la chat in sostituzione del polling attuale.

3.6 Sprint Retrospective

Durante la retrospective il team ha analizzato gli aspetti positivi e negativi dello sprint.

3.6.1 Aspetti positivi

- ottima collaborazione tramite Pull Request e revisione del codice;
- parallelismo efficace nello sviluppo grazie a branch separati;
- buona coerenza visiva dell'interfaccia mantenuta tra le diverse funzionalità;
- completamento di tutte le user story pianificate per lo sprint;
- risoluzione efficace dei conflitti di merge anche in presenza di modifiche sovrapposte (refactoring frontend durante sviluppo attivo).

3.6.2 Criticità riscontrate

- il refactoring della struttura frontend a metà sprint ha introdotto conflitti che hanno richiesto tempo aggiuntivo per la risoluzione;
- alcuni bug (visualizzazione foto mancanti, bug nel pagamento) sono stati scoperti solo dopo il merge in **main**, evidenziando la necessità di test più sistematici prima del merge;
- la stima delle ore per il pannello admin si è rivelata ottimistica a causa della complessità dell'interfaccia a tre tab.

3.6.3 Pratiche Agile adottate

Il team ha adottato le seguenti pratiche Agile durante lo sprint:

- **Sprint Planning:** all'inizio dello sprint il team ha definito collettivamente le user story da sviluppare, stimato i task e assegnato i volontari per ciascuna attività.
- **Sprint Review:** al termine dello sprint il team ha effettuato una demo delle funzionalità implementate, raccogliendo feedback.
- **Sprint Retrospective:** il team ha analizzato il processo di lavoro per identificare punti di forza e aree di miglioramento.
- **Feature branching e Pull Request:** ogni funzionalità è stata sviluppata su un branch dedicato e integrata tramite Pull Request con revisione del codice, in linea con il principio di continuous integration.

3.6.4 Design Thinking

Il team ha applicato alcuni principi del Design Thinking durante lo sviluppo:

- **Empathize:** le funzionalità sono state progettate tenendo conto delle esigenze dei diversi ruoli (utente, host, amministratore), con interfacce dedicate per ciascuno.
- **Prototype:** le schermate principali sono state prototipate iterativamente, con aggiustamenti UI basati sull'utilizzo reale durante i test manuali.
- **Test:** ogni funzionalità è stata testata manualmente prima del merge, con test case documentati nella sezione dedicata.

3.6.5 Dinamiche di gruppo

Il team ha mantenuto una collaborazione efficace per tutta la durata dello sprint:

- la comunicazione è avvenuta in modo costante tramite messaggistica istantanea e incontri periodici di allineamento;
- la suddivisione dei task è stata bilanciata, con ciascun membro responsabile di aree funzionali distinte;
- la revisione reciproca del codice tramite Pull Request ha favorito la condivisione delle conoscenze tecniche all'interno del team.

4 Sezione Finale

4.1 Diagramma del Deploy

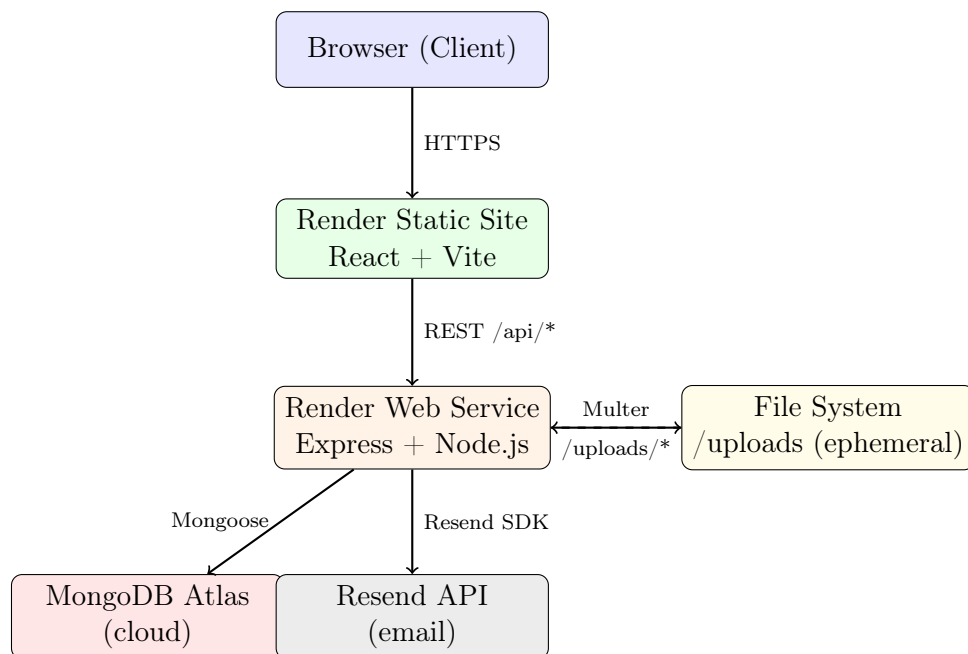


Figure 2: Architettura di deploy di TrentoParking

L'applicazione è deployata su Render ed è composta da tre livelli principali:

- **Frontend:** Static Site su Render (`trentoparking-frontend.onrender.com`). Applicazione React compilata con Vite, comunica con il backend tramite chiamate REST autenticate con JWT.
- **Backend:** Web Service su Render (`trentoparking-backend.onrender.com`). Server Express che espone le API REST, gestisce l'autenticazione JWT, il caricamento file tramite Multer e l'invio email tramite Resend.
- **Database:** MongoDB Atlas (cloud) con Mongoose come ODM. I file caricati vengono salvati nel filesystem del container (`/uploads/`) e serviti come file statici.

4.2 Stack Tecnologico

Backend

Libreria / Framework	Versione	Utilizzo
Node.js	runtime	Runtime JavaScript server-side
Express	4.21.0	Framework HTTP e routing REST
Mongoose	8.7.0	ODM per MongoDB
MongoDB	7.x	Database NoSQL documentale
bcryptjs	2.4.3	Hashing delle password
jsonwebtoken	9.0.2	Autenticazione stateless JWT
Multer	2.1.1	Upload file multipart (foto posti)
Resend	—	Invio email transazionale (verifica account, reset password)
dotenv	16.4.5	Gestione variabili d'ambiente
cors	2.8.6	Cross-Origin Resource Sharing

Table 5: Stack tecnologico backend

Frontend

Libreria / Framework	Versione	Utilizzo
React	19.2.4	Libreria UI dichiarativa
React DOM	19.2.4	Rendering React nel browser
Vite	8.0.1	Build tool e dev server
Leaflet	1.9.4	Mappa interattiva
React-Leaflet	5.0.0	Binding React per Leaflet
lucide-react	1.14.0	Libreria icone SVG

Table 6: Stack tecnologico frontend

Infrastruttura di deploy

Servizio	Piano	Utilizzo
Render Static Site	Free	Hosting frontend React compilato
Render Web Service	Free	Hosting backend Node.js/Express
MongoDB Atlas	M0 Free	Database NoSQL cloud
Resend	Free	Invio email transazionali

Table 7: Infrastruttura di deploy

4.3 Conclusioni

Il secondo sprint ha permesso di trasformare TrentoParking da un prototipo di autenticazione e visualizzazione mappa in un'applicazione web completa per il noleggio di posti auto privati.

Le funzionalità principali — prenotazione con pagamento mock, caricamento foto, recensioni, chat utente-host, gestione posti da parte dell'host e pannello di amministrazione — sono state implementate e integrate con successo. La struttura del codice è stata riorganizzata in `pages/`, `features/` e `components/common/` per favorire la manutenibilità.

Il team ha dimostrato una buona capacità di lavorare in parallelo su branch separati e di integrare le modifiche tramite Pull Request, gestendo anche situazioni di conflitto complesse come quelle introdotte dal refactoring del frontend. Le criticità emerse — in particolare la gestione dei merge e la stima delle ore per funzionalità ad alta complessità di interfaccia — hanno fornito spunti concreti per migliorare il processo di sviluppo.